

דוח מעבדה מספר 8

Databases, ORM, JPA, Hibernate

מערכת ניהול קורסים באוניברסיטה באמצעות Java, Maven, MySQL ו-Hibernate ORM

שם מלא	תעודת זהות	תפקיד בהגשה
Fadi Kees	324222512	מגיש 1
Yousef Salem	213537459	מגיש 2

מטרת הדוח

דוח זה מרכז את המענה למטלות התאורטיות של מעבדה 8 ואת ההסבר על המימוש המעשי של מערכת ניהול הקורסים. הדוח נכתב בהתאם לדרישות ההגשה: קובץ PDF הכולל תשובות למטלות א' ו-ב', ובנוסף קובץ JAR בר הרצה הכולל גם את קובצי המקור.

במימוש השתמשנו ב-Hibernate כדי לבצע מיפוי בין מחלקות Java לבין טבלאות במסד נתונים רלציוני מסוג MySQL. בסיס הנתונים שהוגדר בפרויקט נקרא myFirstDataBase, בהתאם לדרישות התרגיל.

תקציר המימוש

- נבנו ארבע ישויות עיקריות: Student ו-CourseClass, Lecturer, Course.
- הוגדרו קשרים מתאימים בין הישויות: רבים-לרבים בין סטודנטים לקורסים, רבים-לאחד בין קורס למרצה, ורבים-לאחד בין שיעור לקורס.
- נוצרו נתוני התחלה: ארבעה מרצים, ארבעה קורסים, שמונה סטודנטים ושנים עשר שיעורים.
- כל סטודנט משויך לפחות לשני קורסים, וכל קורס כולל שלושה שיעורים.
- התוכנית מדפיסה למסך את כל הרשימות שנדרשו: קורסים, סטודנטים, שיעורים לפי קורסים, ומרצים עם מספר הקורסים שלהם.

מטלה אי' - **ORM, Hibernate** ואנוטציותמהו התפקיד של **ORM**?

ORM הוא קיצור של Object-Relational Mapping, כלומר מיפוי אובייקטי-רלציוני. התפקיד שלו הוא לחבר בין עולם התכנות מונחה העצמים לבין עולם מסדי הנתונים הרלציוניים. במקום לעבוד ישירות מול טבלאות, שורות ופקודות SQL בכל פעולה, המתכנת עובד עם אובייקטים ומחלקות Java רגילות, ו-Hibernate אחראי לתרגם את הפעולות האלה לשמירה, שליפה ועדכון של נתונים במסד הנתונים.

לדוגמה, אובייקט מסוג Student נשמר בטבלת students, והשדות שלו כמו studentNumber, firstName, lastName ו-email נשמרים כעמודות בטבלה. קשרים בין אובייקטים, כמו הקשר בין Student ל-Course, מתורגמים לקשרים בין טבלאות במסד הנתונים.

מתי כדאי להשתמש ב-**ORM** ולא לעבוד ישירות עם **SQL**?

כדאי להשתמש ב-ORM כאשר המערכת בנויה סביב ישויות ברורות וקשרים ביניהן, וכאשר רוב הפעולות הן פעולות רגילות של יצירה, שמירה, שליפה ועדכון. במקרים כאלה ORM מקטין את כמות הקוד הידני, מונע כפילות, שומר על קוד קריא יותר, ומאפשר לעבוד עם אובייקטים במקום לבנות שאילתות SQL רבות באופן ידני.

- מערכות שבהן יש מודל דומיין ברור, למשל מערכת אוניברסיטאית עם סטודנטים, מרצים, קורסים ושיעורים.

- מערכות שבהן יש קשרים רבים בין ישויות וחשוב לשמור אותם בצורה מסודרת.

- פרויקטים שבהם רוצים להפריד בין הלוגיקה של התוכנית לבין פרטי מסד הנתונים.

עם זאת, במקרים שבהם נדרשות שאילתות מורכבות מאוד, אופטימיזציות ביצועים מיוחדות או עיבוד כבד ברמת מסד הנתונים, ייתכן שיהיה נכון להשתמש גם ב-SQL ישיר. לכן ORM אינו מחליף לגמרי SQL, אלא מספק שכבת עבודה נוחה עבור רוב פעולות השמירה והשליפה.

דוגמה לאנוטציה שאינה מובנית ב-Java

אנוטציות של JPA/Hibernate אינן אנוטציות מובנות רגילות של Java כמו @Override, אלא מגיעות מספריות חיצוניות ומשמשות לתיאור האופן שבו מחלקות ושדות יישמרו במסד הנתונים.

- @Entity - מסמנת שמחלקת Java היא ישות ש-Hibernate צריך לנהל ולשמור במסד הנתונים.

- @Table(name = "students") - מגדירה את שם הטבלה המתאימה לישות.

- @Id ו-@GeneratedValue - מגדירות מפתח ראשי ומספר רץ.

- @ManyToOne, @ManyToMany ו-@OneToMany - מגדירות קשרים בין ישויות.

```
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
}
```

מטלה ב' - סוגי קשרים בין ישויות

קשר יחיד ליחיד - One-to-One

קשר יחיד ליחיד הוא קשר שבו רשומה אחת בטבלה אי מקושרת לכל היותר לרשומה אחת בטבלה ב', וגם להפך. כלומר לכל מופע בצד אחד יש מופע אחד מתאים בצד השני.

דוגמה: סטודנט וכרטיס סטודנט. לכל סטודנט יש כרטיס סטודנט אחד, וכל כרטיס שייך לסטודנט אחד בלבד. ב-JPA ניתן לממש קשר כזה באמצעות @OneToOne, ובדרך כלל לשמור מפתח זר באחת הטבלאות.

קשר יחיד לרבים - One-to-Many

קשר יחיד לרבים הוא קשר שבו מופע אחד בצד הראשון יכול להיות קשור למספר מופעים בצד השני, אבל כל מופע בצד השני שייך למופע אחד בלבד בצד הראשון.

דוגמה מתוך התרגיל: מרצה אחד יכול להיות אחראי על כמה קורסים, אך כל קורס מנוהל על ידי מרצה אחראי אחד בלבד. לכן בצד של Lecturer אפשר להגדיר @OneToMany, ובצד של Course מגדירים @ManyToOne עם עמודת מפתח זר lecturer_id.

```
Lecturer 1 ---- * Course
Course.lecturer_id -> Lecturer.id
```

קשר רבים לרבים - Many-to-Many

קשר רבים לרבים הוא קשר שבו כל מופע בצד אחד יכול להיות קשור למספר מופעים בצד השני, וכל מופע בצד השני יכול להיות קשור למספר מופעים בצד הראשון.

דוגמה מתוך התרגיל: סטודנט יכול להירשם למספר קורסים, וקורס אחד יכול לכלול מספר סטודנטים. במסד נתונים רלציוני לא מממשים קשר כזה ישירות בתוך שתי הטבלאות, אלא מוסיפים טבלת קישור שמכילה את המפתחות של שני הצדדים.

• students - טבלת סטודנטים.

• courses - טבלת קורסים.

• student_courses - טבלת קישור עם student_id ו-course_id.

```
@ManyToMany
@JoinTable(
    name = "student_courses",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id")
)
```

הסבר המימוש בפרויקט

מבנה הישויות

הפרויקט מומש במבנה Maven רגיל. מחלקת ההרצה הראשית היא App, מחלקות הישויות נמצאות תחת package בשם org.example.entities, מחלקת DataSeeder אחראית ליצירת נתוני ההתחלה, ו-HibernateUtil אחראית לטעינת הגדרות Hibernate ולבניית SessionFactory.

- Student - מכילה מזהה רץ, מספר סטודנט, שם פרטי, שם משפחה, אימייל ורשימת קורסים שאליהם הסטודנט רשום.
- Lecturer - מכילה מזהה רץ, שם פרטי, שם משפחה, תחום התמחות, אימייל ורשימת הקורסים שבאחריות המרצה.
- Course - מכילה מזהה רץ, שם קורס, קוד קורס, מספר נקודות זכות, מרצה אחראי, רשימת סטודנטים ורשימת שיעורים.
- CourseClass - מייצגת שיעור בפועל. היא מכילה מזהה רץ, תאריך ושעה מסוג LocalDateTime, משך בדקות, קורס ומיקום פיזי של הכיתה.

בחרנו בשם CourseClass במקום Class כדי שהשם בקוד יהיה ברור ולא יבלבל עם java.lang.Class. מבחינת המערכת והמסד, זו עדיין הישות שמייצגת שיעור כפי שנדרש בתרגיל.

מימוש הקשרים בין הישויות

הקשרים הוגדרו בעזרת אנוטציות JPA/Hibernate כדי ש-Hibernate ייצור את הטבלאות ואת המפתחות הזרים בצורה אוטומטית.

- Student מול Course - קשר @ManyToMany באמצעות טבלת student_courses. המתודה addCourse דואגת לעדכן את שני הצדדים של הקשר בזיכרון.
- Course מול Lecturer - בצד Course הוגדר @ManyToOne עם @JoinColumn(name = "lecturer_id", כי לכל קורס יש מרצה אחד. בצד Lecturer קיימת רשימת קורסים עם @OneToMany(mappedBy = "lecturer").
- Course מול CourseClass - בצד Course קיימת רשימת שיעורים עם @OneToMany(mappedBy = "course", cascade = CascadeType.ALL, orphanRemoval = true). בצד CourseClass יש @ManyToOne אל Course.

שימוש ב-cascade בקשר בין קורס לשיעורים מאפשר לשמור את השיעורים כאשר שומרים את הקורס, משום שהשיעורים הם חלק מהמידע של הקורס במערכת.

יצירת נתוני התחלה

במחלקה DataSeeder נוצרו כל הנתונים הנדרשים לפי ההנחיות. הנתונים נשמרים בתוך טרנזקציה אחת, ולאחר כל יחידת עבודה מרכזית מתבצע flush כדי לעדכן את מסד הנתונים בצורה מסודרת.

- ארבעה מרצים: לכל מרצה שם, תחום התמחות ואימייל.
- ארבעה קורסים: לכל קורס שם, קוד, מספר נקודות זכות ומרצה אחראי.
- שנים עשר שיעורים: לכל קורס נוצרים שלושה שיעורים בתאריכים שונים ובמיקום פיזי.
- שמונה סטודנטים: לכל סטודנט יש מספר סטודנט, שם ואימייל.
- כל סטודנט משויך לפחות לשני קורסים, וחלק מהסטודנטים משויכים לשלושה קורסים.

הדפסות, הגדרות והרצה

ההדפסות שמבוצעות בתוכנית

לאחר יצירת הנתונים ושמירתם במסד הנתונים, התוכנית פותחת טרנזקציה נוספת לצורך שליפה והדפסה. ההדפסה מבוצעת דרך CriteriaQuery כדי לשלוף את כל הרשומות מכל ישות בצורה כללית ונקייה.

- רשימת קורסים - כולל מזהה, שם קורס, קוד קורס, נקודות זכות ושם המרצה האחראי.
- רשימת סטודנטים - כולל מזהה, מספר סטודנט, שם מלא, אימייל ורשימת הקורסים שאליהם הסטודנט רשום.
- רשימת שיעורים לכל קורס - כולל תאריך ושעה, משך השיעור ומיקום הכיתה.
- רשימת מרצים - כולל מזהה, שם מלא, תחום התמחות, אימייל ומספר הקורסים שכל מרצה אחראי עליהם.

הגדרות **Hibernate** ומסד הנתונים

קובץ ההגדרות hibernate.properties נמצא תחת src/main/resources. הוא מגדיר את החיבור ל-MySQL, את יצירת הטבלאות ואת שם בסיס הנתונים.

```
hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
hibernate.connection.url=jdbc:mysql://localhost:3306/myFirstDataBase?...
hibernate.hbm2ddl.auto=create
hibernate.connection.username=root
hibernate.connection.password=password
hibernate.connection.timezone=UTC
```

הוספנו שימוש ב-MySQL8Dialect והגדרות timezone כדי למנוע בעיות חיבור נפוצות, במיוחד בסביבת Windows. בנוסף, הוגדר createDatabaseIfNotExist=true בכתובת החיבור, כך שאם למשתמש יש הרשאות מתאימות, מסד הנתונים ייווצר אוטומטית.

אפשרות לשנות סיסמה בזמן הרצה

כדי שכל בודק יוכל להריץ את הפרויקט בלי לערוך את קוד המקור, מחלקת HibernateUtil מאפשרת להעביר פרמטרים בזמן הרצה. החשוב ביותר הוא --db-password, שמחליף את סיסמת ברירת המחדל בקובץ ההגדרות.

```
java -jar target/lab8-orm-university.jar --db-password=YOUR_PASSWORD
```

קיימים גם פרמטרים נוספים כמו --db-url, --db-user ו--db-name, אך עבור ההגשה הרשמית שם בסיס הנתונים נשאר myFirstDataBase.

ניהול טרנזקציות וסגירת משאבים

התוכנית פותחת SessionFactory ו-Session, מתחילה טרנזקציה, שומרת את הנתונים, מבצעת commit במקרה של הצלחה, ומבצעת rollback במקרה של חריגה. בסיום ההרצה ה-session נסגר כדי לא להשאיר חיבור פתוח למסד הנתונים.

בדיקת התאמה לדרישות התרגיל

סטטוס	אופן המימוש בפרויקט	דרישה בקובץ ההנחיות
בוצע	מומשו Student, Lecturer, Course CourseClass-1. הישות CourseClass מייצגת שיעור, והטבלה במסד נקראת classes.	ארבע ישויות: Student, Lecturer, Course, Class
בוצע	מומש באמצעות טבלת קישור בשם student_courses בין students לבין courses.	קשר רבים לרבים בין סטודנטים לקורסים
בוצע	בישות Course נשמר מרצה אחראי יחיד באמצעות מפתח זר lecturer_id.	כל קורס מנוהל על ידי מרצה אחד
בוצע	בישות CourseClass נשמר הקורס שאליו השיעור שייך באמצעות מפתח זר course_id.	כל שיעור שייך לקורס אחד
בוצע	מחלקת DataSeeder יוצרת בדיוק את נתוני ההתחלה הנדרשים.	יצירת ארבעה מרצים, ארבעה קורסים, שמונה סטודנטים ושנים עשר שיעורים
בוצע	נתוני ההתחלה משייכים כל סטודנט לשני קורסים לפחות.	כל סטודנט רשום לפחות לשני קורסים
בוצע	לכל קורס נוצרים שלושה שיעורים, ולכן הדרישה מתקיימת.	כל קורס כולל לפחות שני שיעורים
בוצע	התוכנית מדפיסה קורסים, סטודנטים, שיעורים לפי קורסים ומרצים עם מספר הקורסים.	הדפסות נדרשות למסך
בוצע	שם בסיס הנתונים מוגדר בכתובת החיבור בקובץ hibernate.properties.	שם בסיס הנתונים myFirstDataBase
בוצע	ניתן להעביר סיסמה בזמן הרצה באמצעות הפרמטר db-password בלי לשנות קוד.	אפשרות לשנות סיסמת מסד נתונים

סיכום

המימוש עומד בדרישות התרגיל: הוא משתמש ב-Hibernate ORM, מגדיר את כל הישויות והקשרים הנדרשים, יוצר נתוני התחלה תקינים, שומר אותם במסד הנתונים ומדפיס את כל הרשימות שהתבקשו. בנוסף, שם בסיס הנתונים הוגדר כ-myFirstDataBase ונוספה אפשרות לשנות את סיסמת החיבור בזמן הרצה, כך שהבודק יוכל להריץ את התרגיל בסביבה שלו בצורה נוחה.

GitHub - קישור לקוד המקור ב-GitHub

בנוסף לקובץ ה-JAR שמוגש לפי הוראות המעבדה, ניתן לצפות בקוד המקור המלא גם בענף GitHub של הפרויקט. הקישור מיועד לנוחות הבדיקה בלבד, וההגשה הרשמית נשארת קובץ PDF יחד עם קובץ JAR בר-הרצה הכולל את קבצי המקור.

GitHub branch: https://github.com/FadiKais1/Software_Engineering_Course-Labs/tree/LAB-8-ORM

בענף זה נמצאים קבצי הפרויקט, כולל מחלקות הישויות, הגדרות Hibernate, מחלקות יצירת הנתונים, מחלקות ההדפסה וקובצי Maven הדרושים לבנייה והרצה של התרגיל.